jigsaw-agile-community-rules

6th Place Solution: Online Distillation via Deep Mutual Learning

Rank	6
Team	ducnh279
Author	ducnh279
Topic ID	613150
Version	Original

Source: https://www.kaggle.com/competitions/jigsaw-agile-community-rules/discussion/613150
Retrieved with Kaggle CLI on 2026-10-21.

First, I'd like to thank the organizers for hosting such an amazing LLM competition. Huge thanks also to my teacher, @justmarkham - your online data science education has been invaluable, and I couldn't have done this without you.

Summary

For this competition, I fine-tuned 3 LLMs from the Qwen3 family for text classification using **Deep Mutual Learning (DML)**, and applied weighted ensembling to improve robustness and stability.

Input Data

Test positive and negative examples, along with their rules, were used for training and formatted into the following prompt template:

```
prompt = f"""<|im_start|>user
You are given a Reddit comment and a specific subreddit rule. Your task is to decide if the comment
violates the rule. Respond only with "Yes" or "No".

Rule: {RULE_HERE}

Now, here is the comment to classify:
"{COMMENT_HERE}"

Answer "Yes" if it violates the rule, otherwise "No".<|im_end|>
<|im_start|>assistant
<think>

</think>

Answer:"""
```

Models

- **Models for experimentation:** `Qwen3-14B` and `Qwen2.5-14B-Instruct`
- **Models for final submission:** `Qwen3-14B`, `Qwen3-8B`, and `Qwen3Guard-Gen-4B`

Early Competitive Submission

Training single Qwen3-14B on test examples, quickly get

Model	Public Score	Private Score
Qwen3-14B	0.925	0.921

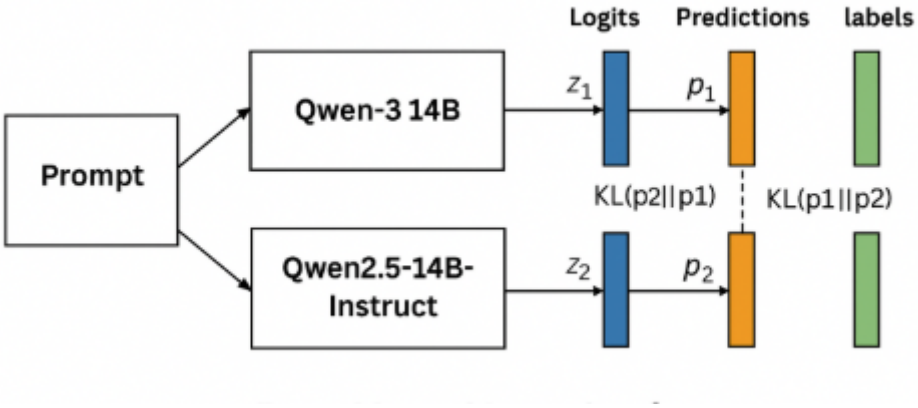
Deep Mutual Learning

Paper: Deep Mutual Learning (arXiv:1706.00384)

With only 2 T4 GPUs, using a larger teacher model to improve the 14B model's performance is not feasible. Offline distillation could be attempted, but it only covers 2 out of the 6 known rules in the training set. To achieve the best overall score, knowledge distillation needs to incorporate all rules.

Classic logits-based knowledge distillation poses a challenge in this competition because the teacher's softmax outputs are extremely sharp, almost identical to one-hot labels. Applying temperature scaling to soften them is ineffective.

Under these constraints, a more practical approach is **Deep Mutual Learning (DML)**, an online distillation method where multiple peer student models of similar or smaller size learn from each other simultaneously. This enables effective knowledge transfer without a larger teacher, simply by adding a **Kullback-Leibler (KL) divergence loss** to the standard classification cross-entropy.



The advantage of DML is that it removes the need for a larger and stronger teacher by allowing models to learn from each other's soft predictions rather than hard labels. This process enriches supervision, adds implicit regularization, and encourages smoother decision boundaries. Each peer model effectively gains extra capacity from the knowledge shared by the other, leading to more stable optimization and better generalization. This makes DML especially valuable in LLM competitions or modern industry environments, where hardware constraints often prevent training or deploying a large teacher model, yet effective knowledge transfer and robust performance remain essential.

Effect of DML on Qwen3-14B & Qwen2.5-14B Performance

Peer Students: Qwen3-14B, Qwen2.5-14B-Instruct

Training Type	Model	Public Score	Private Score
Independent	Qwen3-14B	0.925	0.921

Training Type	Model	Public Score	Private Score
Independent	Qwen2.5-14B-Instruct	0.923	0.919
DML	Qwen3-14B	**0.929**	**0.925**
DML	Qwen2.5-14B-Instruct	**0.929**	**0.924**

Ensembles

Ensemble Type	Public Score	Private Score
DML models	**0.932**	**0.9271**
Independently trained models	0.929	0.925

DML consistently improves performance for both Qwen3-14B and Qwen2.5-14B, with notable gains in private scores and ensemble results, demonstrating its effectiveness in boosting model generalization.

Final Submission

While DML adds extra capacity to each model, its co-training process can introduce correlations in the models' errors. Since diversity is crucial for a strong ensemble, 2 days before the deadline, I decided to create a cohort of 3 models for DML. This approach enhances ensemble diversity and further helps mitigate the one-off mistakes of each "peer" model.

Performance of the 3-peer ensemble

Peer Students: `Qwen3-14B`, `Qwen3-8B`, `Qwen3Guard-Gen-4B`

Public Score	Private Score
0.93237	0.92781



Model Execution Time

- **Training:** Three models were trained together in a single DML process, taking about 5 hours.
- **Inference:** After training, each model was run sequentially to generate predictions for ensembling. The test data was split in half, keeping the distribution of sequence lengths similar in both halves to balance GPU workloads. Each half was then sorted by length for efficient batch processing and run in parallel across 2 GPUs. The entire inference process took around 5 hours.

Supplemental Q&A

Shivam A. · 2025-10-24T15:15:59.417000

Congratulations! Great Solution. Thanks for sharing it.

Do you have any advice regarding finding optimal configuration (LORA config, learning rate etc.) for LLMs ?

ducnh279 · 2025-10-24T15:37:13.543000

I actually wanted to include this in my post, but decided to keep the focus on the main factors of my solution, so I left it out. Thanks for asking!

Here are some of my personal experiences with LoRA configurations, mainly from fine-tuning the Qwen family. These are just some defaults that have worked well for me:

- **Qwen ≤ 14B** → `lr=2e-4`, `rank=8`, `alpha=16`
- **Qwen > 14B** → `lr=5e-5` (or slightly lower), `rank=8`, `alpha=16`

For classification and regression tasks, I often replace the lm_head with a new head that matches the embedding or hidden dimension. While keeping the original lm_head can be useful for vLLM inference, it requires careful learning rate tuning between the head and the backbone.

In my case, I usually replace the head entirely and use the same learning rate (lr=2e-4) for both parts, no need for differential LRs. This approach also helps reduce memory usage during training, since the original lm_head has a large number of parameters (used for vocabulary next-token prediction/decoding). For more details, I had a related discussion with @conjuring92 for the design choice on `heads`. https://www.kaggle.com/competitions/wsdm-cup-multilingual-chatbot-arena/discussion/567589#3154067

I don't feel confident sharing setups for other model families, but in general, it all comes down to getting reliable validation set and then fine-tuning, adjusting LoRA rank (r), alpha, and learning rate to find the best combination that optimize the val score.

In my experience, variants like MoRA, DoRA, etc. usually perform worse than standard vanilla LoRA.

You might also want to check out this excellent blog by Sebastian Raschka, it's full of practical fine-tuning tips:
Blog: Practical Tips for Fine-Tuning LLMs.

At the end of the day, I think it's still about trying things out and checking your score 😄

Pavel Kazlou · 2025-10-24T16:08:59.290000

Huge respect for such an interesting solution! And congrats for the top place!
In the code I can see that the KL loss is symmetric between the models, despite their different size. Does it make sense to apply some weights to KL losses (to penalize less larger model) in such setups?

ducnh279 · 2025-10-24T16:18:48.550000

Great question!!! I actually experimented with weighted DML, but it performed worse than symmetric DML (with $\lambda_{??} = 1$), so I followed the paper's original approach. One takeaway is that all pretrained peer models should have roughly balanced capacity (they can be slightly better or worse than each other). When this balance is maintained, symmetric DML training becomes more stable, and all peers improve together.

Raviksh Singh Dikola · 2025-10-24T17:13:54.560000

So you have said that you replaced the head of llm model like qwen 14b.

Does that customized model of your is compatible for vllm inferencing framework?

ducnh279 · 2025-10-24T17:19:24.877000

Thanks for your question! I did not use vllm. Maybe with my current setup, it will be not compatible with vllm. But Unsloth has a trick on this topic! You may want to check out the notebook. It shows how to replace the classification head in a way that remains compatible with vLLM inference. Big shoutout to @danielhanchen.

Colab: https://colab.research.google.com/github/timothelaborie/text_classification_scripts/blob/main/unsloth_classification.ipynb#scrollTo=FNLDlj923Fx9

BokingChen · 2025-10-24T17:46:26.537000

Thanks for sharing ! A highly efficient and innovative solution! After reviewing your solution code, does your training only utilize positive and negative examples? Additionally, what is the total time for training and inference for your entire code? It's unbelievable that you managed to use multiple LLM models within a limited amount of time!

ducnh279 · 2025-10-24T17:50:38.723000

> does your training only utilize positive and negative examples

Yes, I ultilize all pos and neg examples for training!

> Additionally, what is the total time for training and inference for your entire code?

Training + inference took around 9-10 hours for all 3 models.

> It's unbelievable that you managed to use multiple LLM models within a limited amount of time!

I used Distributed Data Parallel (DDP) for training and ran inference in parallel on 2 GPUs, ensuring no GPU stayed idle. I simulated 55k samples and optimized the inference pipeline locally.

Nikita Babych · 2025-10-24T17:52:59.263000

Congrats on the solo and your first gold!
Using DML is an interesting approach, but I'm curious how much better Qwen3-14B (DML) with `Peer Students: Qwen3-14B, Qwen2.5-14B` performs compared to ensembling separately trained Qwen3-14B and Qwen2.5-14B.
According to my experiments, you should get similar results. If that's the case, then it mainly comes down to time savings since in both cases you train two models (seems that DML is not parallelized), but with DML you only need to run inference once for Qwen3-14B, right?

p.s. I see that despite mutual training and high correlation, their ensemble still produces a higher score of 0.932 than separate training.

Chew Kok Wah · 2025-10-24T18:10:43.880000

May I know whether the type of quantization of the based model (BnB, GPTQ, AWQ etc) we used will affect the result of the training?

I saw your code is using quantization on the fly into BnB, is it chosen because of your training pipeline doesn't allow the use of other form of pre-quantize model, or you found that BnB provide the best result?

ducnh279 · 2025-10-24T18:27:57.137000

Thanks @nikitababich for your questions! First, I want to share that I was inspired by your solution using Noisy Student Training. I actually tried it in this competition, but unfortunately didn't have much luck with it. Still, I believe that some of the good old techniques and papers from around 2017–2020 can still be valuable and provide a real edge. I've been exploring various techniques to improve my model, and DML is one of them.

> how much better Qwen3-14B (DML) with Peer Students: Qwen3-14B, Qwen2.5-14B performs compared to ensembling separately trained Qwen3-14B and Qwen2.5-14B.

Here are the results comparing **separately trained models** vs **DML-trained models**:

Model	Public Score	Private Score	Notes
Qwen3-14B (trained separately)	**0.925**	**0.921**	—
Qwen2.5-14B (trained separately)	N/A	N/A	Not submitted individually
Ensemble: Qwen3-14B + Qwen2.5-14B	**0.929**	**0.925**	Ensemble of separately trained models
Ensemble: Qwen3-14B + Qwen2.5-14B	**0.932**	**0.927**	Ensemble of DML models

-> So, using DML gives around **+0.003 on Public** and **+0.002 on Private** compared to ensembling the same models trained separately.

> you train two models (seems that DML is not parallelized)

Yeah, DML isn't parallelized since I couldn't get DDP to work with it. During DML training, I didn't split model parameters across GPUs (no device_map='balanced' or 'auto'). Instead, I made sure each model's full parameters fit on a single GPU. This helps reduce data transfer and communication overhead, as PyTorch's CUDA semantics automatically handle device placement efficiently.

> DML you only need to run inference once for Qwen3-14B, right?

Yes one time! I trained a cohort of two/three models! Each epoch takes about 1 hour and 30 minutes, so around 4 hours and 30 minutes in total for training. During inference, I sequentially run each of the peer models in the cohort in parallel across two GPUs and then ensemble their outputs, which takes around 5 hours.

ducnh279 · 2025-10-24T18:34:36.233000

There isn't much difference in performance between training with LoRA + bnb quantized model and LoRA + quantized gptq model. I chose bnb simply because I've been using it since my early days of fine-tuning LLMs and stick with it.

Aic · 2025-10-25T11:52:13.813000

Congratulations on winning the solo gold medal!
I learned a lot from reading your solution.

You mentioned:
`Since diversity is crucial for a strong ensemble, I decided, two days before the deadline, to create a cohort of three models for DML.`

I'd like to ask — when you introduced the third model to enhance diversity, did you perform pairwise mutual learning between the models, or did all three models learn from each other simultaneously during DML and then ensemble afterward?

Also, is there a significant difference between these two approaches in terms of performance? Would the second method (all three models learning together) potentially yield better results?

ducnh279 · 2025-10-25T11:58:13.553000

> when you introduced the third model to enhance diversity, did you perform pairwise mutual learning between the models, or did all three models learn from each other simultaneously during DML and then ensemble afterward?

It's `three models learn from each other simultaneously during DML and then ensemble afterward`

If you have time or interests in DML for more than 2 models, you could read the DML paper and specifically on section `2.3 Extension to Larger Student Cohorts`.

> Also, is there a significant difference between these two approaches in terms of performance? Would the second method (all three models learning together) potentially yield better results?

The larger the number of models in the DML cohort, the better the ensemble performance.

Ankith Savio · 2025-10-25T12:58:46.487000

Thanks for the solutions and tips @ducnh279 . Could you also share your CV strategy? I remember your early post on it and would be nice to know what setup provided incremental results for your experiments

ducnh279 · 2025-10-25T13:14:34.560000

The LB turned out to be more reliable, so I decided to trust it more for this competition. That said, I did use a CV strategy.

1. I combined all positive, negative, and original training samples (body)
2. Drop duplicates based on both rule_comment
   df['rule_comment] = df['rule'] + df['comment']
   df = df.drop_duplicates(['rule_comment'])
3. Perform stratified k-fold splitting by rule_label
   df['rule_label'] = df['rule'] + df['label'].astype(str)

After inference on full validation set, I computed the score separately for each of the two rules and took their average - following the same averaged-AUC metric used in the competition. This CV setup effectively validated my DML approach and transferred well to the LB.

ducnh279 · 2025-10-25T17:53:58.303000

@nikitababich Hi Nikita, I just saw your newly edited comment.

> p.s. I see that despite mutual training and high correlation, their ensemble still produces a higher score of 0.932 than separate training.

You're right, DML tends to make model errors more correlated. However, that doesn't mean ensembling them won't help. When DML is done properly, ensembling models within the same DML cohort typically outperforms an ensemble of independently trained models.

Here's a copy from my older dev notebook using DML between Qwen2.5 and Qwen3-14B. You can fork it and submit, it should give around **0.932x** on Public and **0.927x–0.928** on Private. Pls let me know if the scores are weird compared to what I report.

Notebook: https://www.kaggle.com/code/ducnh279/dml-qwen3-14b-qwen2-5-14b